

IA PARA EL DESARROLLO DE JUEGOS

BLOQUE 1. MOVIMIENTO



Autores:

Mohammed Amrou Labied Nasser, 49857930W
Sergio García García,
Diego Sergio Ballesta González de la Higuera, -

Tutor:

Luis Daniel Hernández Molinero-

Grupo y subgrupo: 1.1

Fecha de entrega:

21-03-2026

Índice

1. Introducción

En esta memoria se documenta el diseño y la implementación del **Bloque 1: Movimiento** de la asignatura Inteligencia Artificial para Videojuegos. El propósito principal de este bloque es el desarrollo de un sistema de navegación reactivo y autónomo (steering behaviors) para diferentes tipos de agentes dentro del motor de desarrollo **Unity**.

A lo largo de este documento, se detallarán los métodos y soluciones aplicados para resolver los desafíos de movilidad planteados en el enunciado. Se explicará cómo se han estructurado los elementos pedidos en el enunciado, algunos elementos fundamentales son los siguientes;

- **Motor de físicas:** Como diseñamos distintos aspectos de nuestro motor; como el movimiento angular, la aceleración, el movimiento rectilíneo, etc. Mediante las ecuaciones de Newton-Euler, operaciones de vectores, transformadas, etc. Con el objetivo de trabajar sobre una dimensión 2.5D.
- **Sistemas de Steering :** Desarrollo de algoritmos para el control de la aceleración y dirección de los agentes, incluyendo comportamientos básicos como Seek y Arrive, y delegados como Wander.
- **Detección y Evitación de Obstáculos:** Implementación de sistemas de colisión basados en rayos (*raycasts*) o "bigotes" para garantizar una navegación fluida en entornos complejos.
- **Gestión de Grupos y Formaciones:** Organización de múltiples unidades bajo estructuras de movimiento coordinado y seguimiento de líderes.
- **Búsqueda de Caminos (Pathfinding):** Aplicación del algoritmo **LRTA*** sobre una representación del escenario basada en rejillas (*grids*) para alcanzar el objetivo.

2. Arquitectura del proyecto

2.1. Esquema de escenas del proyecto

```

Scenes/
|___Arbitros.unity
|___PathFinding
|   |___PathFinding.unity
|___Basic
|   |___Seek_Flee_Arrive.unity
|   |___Align-AntiAlign.unity
|___Delegate
|   |___PathFollowing+Face+Wander.unity
|   |___PathFollowing+Jumping.unity
|   |___WallAvoidance.unity
|___Formaciones
|   |___LeaderFollowing.unity

```

Figura 1: Estructura de escenas del proyecto en Unity

Descripción de las escenas del proyecto:

El proyecto está dividido en varias escenas, cada una diseñada para mostrar y probar diferentes comportamientos de los agentes y técnicas de inteligencia artificial en Unity. A continuación se explica brevemente el propósito de cada una:

- **Arbitros.unity:** Escena dedicada a los agentes árbitros, que combinan varios comportamientos de steering para gestionar situaciones complejas y tomar decisiones en tiempo real.
- **PathFinding/PathFinding.unity:** Aquí se implementa y prueba el algoritmo de búsqueda de caminos (LRTA*) sobre una rejilla, permitiendo a los agentes navegar el escenario y encontrar rutas óptimas evitando obstáculos.
- **Basic/Seek_Flee_Arrive.unity:** Escena para los comportamientos básicos de steering: Seek (perseguir), Flee (huir) y Arrive (llegada suave al destino), mostrando la lógica fundamental de movimiento.
- **Basic/Align-AntiAlign.unity:** Centrada en los comportamientos de alineación, donde los agentes ajustan su orientación para coincidir o contrarrestar la de un objetivo.
- **Delegate/PathFollowing+Face+Wander.unity:** Combina Path Following (seguir un camino), Face (orientarse hacia un objetivo) y Wander (deambular de forma autónoma), mostrando la alternancia entre estos comportamientos.
- **Delegate/PathFollowing+Jumping.unity:** Prueba el seguimiento de caminos junto con la capacidad de los agentes para saltar obstáculos, mostrando una navegación avanzada.

- **Delegate/WallAvoidance.unity:** Dedicada a la detección y evasión de paredes u obstáculos mediante raycasts o bigotes, asegurando movimientos sin colisiones.
- **Formaciones/LeaderFollowing.unity:** Implementa la lógica de formaciones, donde varios agentes siguen a un líder, manteniendo posiciones relativas y desplazándose como un grupo coordinado.

3. Roles de los NPCs

Deben existir 3 NPCs con distintos parametros del body

4. Selección y control de personajes

5. Steering básicos acelerados

5.1. Seek

5.2. Flee

5.3. Arrive

5.4. Leave

5.5. Align

5.6. anti-Align

6. Steering Delegado

6.1. Face

6.2. Wander

6.3. Path Following sin PathOffset

7. Detección de colisiones (WallAvoidance)

8. Steering combinado (Árbitros)

9. Formaciones

9.1. Estructuras Fijas

9.2. Criterio Leader Following

10. LRTA*

LRTA* (*Learning Real-Time A**) es un algoritmo de búsqueda en tiempo real que no necesita calcular la ruta completa antes de moverse. En cada decisión, el agente observa un entorno local, aprende (actualiza su estimación de coste) y ejecuta solo el siguiente movimiento. Este enfoque es especialmente útil en videojuegos porque permite reaccionar de forma continua, con coste computacional acotado por frame.

Formalmente, LRTA* mantiene una función de coste aprendido $h(s)$ para cada celda s . En cada iteración se aplica una actualización local tipo Bellman:

$$h(u) \leftarrow \max \left(h(u), \min_{v \in N(u)} (c(u, v) + h(v)) \right)$$

donde $N(u)$ son los vecinos transitables y $c(u, v)$ es el coste de transición.

10.1. Diseño e implementación aplicada en nuestro proyecto

La implementación se ha organizado por módulos para separar claramente representación del mapa, heurística, construcción del espacio local y control del agente:

- **Representación del entorno** (`Graph.cs`): define la rejilla ($nFilas$, $nColumnas$, $tamCelda$), convierte coordenadas mundo-grid y detecta celdas bloqueadas mediante `Physics.OverlapBox` y etiqueta `OCUPADO`.
- **Heurísticas** (`HeuristicaType` + **implementaciones**): estrategia intercambiable (Manhattan, Chebyshev y Euclídea). Además de estimar distancia, cada heurística define si permite diagonales.
- **Espacio local** (`LssBuilder.cs`): construye el *Local Search Space* (LSS) con un BFS acotado por `maxLSSNodes`.
- **Núcleo LRTA*** (`LRTA.cs`): integra inicialización de costes, actualización de valores, selección de acción y conexión con el steering **Seek** mediante un objetivo virtual.

La estructura interna del flujo en `LRTAStar(start, goal)` sigue tres fases por ciclo:

1. Construcción del LSS local alrededor del estado actual.
2. Aprendizaje de costes en ese LSS (`ValueUpdateStep`).
3. Elección de la mejor acción inmediata (`GetBestNeighbor`).

Con este diseño, el sistema está desacoplado: se pueden cambiar heurísticas, tamaño del LSS o política de recálculo sin rehacer el resto del algoritmo.

10.2. Generación y validación del grafo de búsqueda

Antes de planificar, se validan inicio y objetivo: deben estar dentro de la rejilla y no ser celdas ocupadas. El coste aprendido se almacena en `cellCosts[row,col]`:

- Celdas bloqueadas $\rightarrow +\infty$.
- Celdas transitables $\rightarrow h$ inicial según heurística elegida.

El vecindario se construye con `GetTraversableNeighbours`. Si la heurística no permite diagonal (caso Manhattan), solo se usan movimientos ortogonales. Si hay diagonal (Chebyshev/Euclídea), se evita *corner cutting*, es decir, atravesar una esquina entre dos bloques.

El coste de movimiento usado es:

$$c(u,v) = \begin{cases} 1 & \text{si el movimiento es ortogonal} \\ 2 & \text{si el movimiento es diagonal} \end{cases}$$

10.3. LRTA* un paso (fase de preparación)

Para verificar que el aprendizaje era correcto, primero trabajamos con una versión de comportamiento equivalente a **1 paso de decisión por iteración**: el agente calcula, aprende y avanza solo al mejor vecino. En esta fase observamos el patrón típico de LRTA*: en zonas ambiguas, el NPC puede avanzar y retroceder temporalmente hasta que los costes aprendidos penalizan la ruta mala.

Ese comportamiento de “ida y vuelta” fue clave para validar que el aprendizaje estaba activo: cuando una zona intermedia del mapa acumula coste alto por repetición (máximo local de coste aprendido), el agente deja de insistir en ese corredor y estabiliza el avance por la alternativa correcta.

En nuestra implementación, esta dinámica se ve reforzada porque se puede recalcular en cada celda (`recalculateEachCell`), y porque la acción final siempre se toma tras actualizar valores con `ValueUpdateStep`.

10.4. LRTA* con n-pasos y espacio local (LSS)

Una vez validado el caso base, ampliamos a LRTA* con **n-pasos de exploración local** mediante LSS. El parámetro `maxLSSNodes` controla cuántos nodos del entorno se considerarán antes de actuar:

- Si `maxLSSNodes` es pequeño, el comportamiento se aproxima al LRTA* más reactivo (muy local).
- Si `maxLSSNodes` aumenta, el aprendizaje por iteración es más informativo y reduce oscilaciones.

En términos prácticos, este espacio local mejora la eficiencia global porque evita “re-aprender” lo mismo en cada celda con información mínima. En cada ciclo se actualiza un bloque de estados vecinos (no solo el estado actual), de forma que la política mejora más rápido con menos pasos físicos del NPC.

10.5. Ejemplo ficticio del cálculo de n-pasos

Supongamos un mapa de 7×7 , inicio en $(1, 1)$, meta en $(5, 5)$ y un obstáculo central que obliga a rodear. Si fijamos `maxLSSNodes = 6`:

1. El BFS del LSS toma 6 celdas alcanzables desde $(1, 1)$.
2. En esas 6 celdas, se aplica la actualización: $h(u) = \min_{v \in N(u)} (c(u, v) + h(v))$ (sin disminuir por debajo del valor ya aprendido).
3. Después se selecciona la mejor acción desde el estado actual:

$$a^* = \arg \min_{v \in N(s)} (c(s, v) + h(v))$$

4. El agente avanza solo a ese vecino y repite el proceso en la siguiente iteración.

Si en una iteración el frente local detecta que seguir “hacia el obstáculo” implica valores crecientes, el coste aprendido de esa zona sube y en las siguientes decisiones el algoritmo prefiere bordear el bloqueo. Esa es la razón de que, con n-pasos, el comportamiento sea más estable y converja antes que en el caso puramente de 1 paso.

10.6. Heurísticas

La selección de heurística se realiza mediante `enum` y un `switch` en tiempo de inicialización:

10.6.1. Manhattan

$$h(n) = |x_n - x_g| + |y_n - y_g|$$

No permite diagonales y es coherente con movimientos ortogonales en rejilla.

10.6.2. Chebychev

$$h(n) = \max(|x_n - x_g|, |y_n - y_g|)$$

Permite diagonales y modela bien desplazamientos en 8 direcciones con coste uniforme aproximado.

10.6.3. Euclídea

$$h(n) = \sqrt{(x_n - x_g)^2 + (y_n - y_g)^2}$$

Permite diagonales y aporta una estimación geométrica directa de la distancia al objetivo.

10.7. Integración con steering y depuración

La salida de LRTA* se integra con el movimiento del NPC mediante un objetivo virtual, que alimenta el comportamiento `Seek`. Así se desacopla el razonamiento discreto (grid) del movimiento continuo del agente en mundo 3D.

Además, se implementaron mecanismos de depuración visual muy útiles durante el desarrollo:

- Dibujo del último LSS en verde semitransparente.
- Dibujo del último tramo de camino calculado.
- Visualización de costes aprendidos por celda (*wire cubes* + etiqueta numérica).

Esta instrumentación permitió comprobar en tiempo real cómo evolucionaban los valores aprendidos y por qué el agente cambiaba de decisión en cada iteración.

11. Mecanismo de depuración

12. Conclusión